

Ghana STEM Trivia — Build Log, Roadblocks & Solutions (Phase 1 → Hosting)

Author: Stephen Barnie Amoako (Peswa)

Last updated: 2026-08-19

Project: `ghana-stem-trivia` (local + temporary public hosting)

0. Summary

Ghana STEM Trivia is a mobile-first, curriculum-aligned (NaCCA, B4–B9) STEM quiz web app. This document records the work done from the initial build through to a **publicly-hosted, testable build**, including every roadblock hit and how it was resolved. It is written as an honest engineering log, not a polished press release: where something failed, that failure is recorded.

Key outcomes:

- Backend + 669 verified questions, running on FastAPI (port 8001).
- Frontend: glassy, animated React PWA with two Ghanaian-learner mascots.
- A **forgot-password** flow (security-question based, no email).
- A **per-question speed-bonus** scoring fix (early answers rewarded, slow answers never penalised).
- **Temporary public hosting** via a Cloudflare quick tunnel (single-origin app).

> ■■ **Hosting is TEMPORARY.** The public link is a Cloudflare quick tunnel that
> dies when this machine sleeps or the process stops. It is for testing only.

1. Foundation-first build (recap)

Per the project's engineering discipline, the backend and a 200-question seed were built and verified **before** any UI was written. The question bank was later expanded and vetted to **669 clean questions** (see §4).

Stack:

- Frontend: React 18 + Vite 5 (`vite-plugin-pwa`), Framer Motion.
- Backend: FastAPI + Uvicorn, SQLite (`trivia.db`), JWT auth (registered + guest).
- Animations: Framer Motion; mascots are animated SVG (no image assets).

2. UX redesign (glassy + animated)

Goal: modernise the UI without touching backend behaviour.

****What was done****

- Typography: Space Grotesk (display) + Inter (body).
- Deep gradient background + three floating blurred "blobs" (`AnimatedBg.jsx`).
- Glassmorphism cards (`backdrop-filter: blur`), spring-physics page transitions.
- Kente-inspired frame accent (`KenteFrame.jsx`).
- Two animated Ghanaian-learner mascots (navy uniform, Kente badge, cornrows/low-cut).
- Redesigned speech bubble (`Mascot.css`) with class-aware intensity (B4–B6 bouncier).

****Roadblock — blank landing page (the big one)****

After editing `Landing.jsx` (security-question + reset flow) the page rendered completely blank. Investigation:

- The whole app was blank (not just the landing) → crash at the top of the React tree.
- SSR of each component in isolation passed, so no module-level throw.
- Vite build passed, so no syntax error.
- The tab title briefly showed `ERR: Unc...` → a runtime `Uncaught` error.

****Root cause:**** `Landing.jsx` referenced `api.securityQuestions` but had ****never** imported `api`******. Two compounding mistakes were made while fixing:

1. Added `import api from '../api.js` (default import) — but `api.js` uses a ****named export**** `export const api`, so it threw `does not provide an export named 'default'`.
2. Corrected to `import { api } from '../api.js`.

****Lesson:**** always match the export style (`named` vs `default`). The bug was a single missing import line — trivial to fix, painful to find without reading the exact tab title.

****Resolution:**** import fixed, build clean, page renders. Verified by hard-reload (Ctrl+Shift+R) after clearing the Vite cache.

3. Forgot-password + data hygiene

****Request:**** add forgot-password, fix bubble overlap, redraw mascots as Ghanaian students, and wipe all student/guest accounts + derived data (keep admins).

****What was done****

- `models.py`: added `security\_question` + `security\_answer\_hash` columns.
- `database.py`: idempotent `ALTER TABLE` migration for the new columns.
- `schemas.py`: `RegisterRequest` now carries the security question; added `ResetQuestionRequest` / `ResetVerifyRequest`.
- `auth.py`: `POST /auth/reset/question` (returns whether a user has a question, with enumeration guard) and `POST /auth/reset/verify` (validates the answer,

returns a fresh JWT). Answers are hashed lowercase.

- `Landing.jsx`: signup security-question dropdown + "Forgot password?" inline flow.
- `Mascot.jsx`: redrawn as a Ghanaian school pupil; bubble moved **above the head**

(sibling element) to remove the overlap.

- `scripts/clear\_users.py`: backs up `trivia.db`, deletes all student/guest rows

and derived data, **keeps the 4 admin users and the 669 questions**.

Verification (API, end-to-end):

Step	Result
Register with security question	200
Fetch the user's question	correct question returned
Wrong answer	401 rejected
Correct answer	200 + auto-login token
Login with new password	200
Old password after reset	401 fails
Unknown nickname	generic `has_security_question: false` (no leak)

Roadblock — stale/cached localhost tab

During visual verification, the browser kept showing an **old cached build** of the landing page (two side mascots, no walking hero) even after code changes. Cause: multiple localhost tabs + a stale module cache. Resolution: restarted Vite, cleared `node\_modules/.vite`, and hard-reloaded the correct tab.

4. Walking mascots + letter interaction + bubble behaviour

Request: (a) remove the exposed admin password from the login screen, (b) make the mascots **walk around and interact with the "Ghana STEM Trivia" letters**, (c) the thought bubble should **not stay forever** and should be **hidden while walking**.

What was done

- `Admin.jsx`: removed the hint that printed `admin / Admin@1234` on the login screen (security improvement — the password was literally displayed).

- `HeroStage.jsx` (new): the title is split into individual animated **letter** spans (spring-in + happy hop). Two mascots **walk back and forth** along a track with a leg-swing walk cycle; as each passes a letter, that letter does a pop/hop.

- `Mascot.jsx`: added a `walking` prop. While walking, the speech bubble is **hidden**. When idle/state changes, the bubble appears briefly (~4s) then **auto-hides** (previously it stayed on screen permanently).

Verification: `vite build` passes (exit 0); `HeroStage` renders via SSR with

no error (8227 chars); all framer-motion hooks confirmed present (v13). Dev server confirmed serving the new code.

5. Scoring fix — reward early answers, never punish

Report: "if a question is not answered early it makes the user start again; it shouldn't be so — just give extra points if answered early."

Investigation: searched the entire frontend + backend for any timer, auto-advance, or restart logic. **None exists** — there is no per-question timer and nothing forces a restart. The real issue was the **scoring model**:

- Old rule used the **quiz-average** time, applied uniformly to every question.

So answering early vs late within a quiz made no difference.

- Speed bonus was tiny (+5 max) and based on average — weak incentive.

What was done

- `schemas.py`: `AnswerSubmission` gained `seconds_taken` (per-question time).
- `scoring.py`: rewrote `question_points` to a **per-question** speed bonus:

- ≤ 6 s $\rightarrow$ full bonus (**+10**)

- 6–30 s $\rightarrow$ linear falloff

- ≥ 30 s $\rightarrow$ **base points only**, never below base (no penalty, no restart)

- `quiz.py`: `submit_result` now scores each answer by **its own** time instead of the quiz average.

- `Quiz.jsx`: tracks per-question start time, sends `seconds_taken`, and shows a

non-punishing `Ns` timer (purely cosmetic; nothing auto-advances).

Verification (unit + end-to-end through the API):

Case	Points	Expected
Easy @2 s (fast)	20	10 + 10 ■
Hard @2 s (fast)	35	25 + 10 ■
Medium @18 s (mid)	20	15 + 5 ■
Medium @40 s (slow)	15	base only, never penalised ■

Outcome: early answers earn extra points; slow answers keep full base score; the quiz never restarts. This directly satisfies the request.

6. Public hosting on Cloudflare (temporary)

Request: combine frontend + backend and host on Cloudflare for people to try, as a free temporary link.

****Roadblock 1 — 403 from Cloudflare's edge****

First attempt: ``vite preview`` (``:4173``) for the frontend + backend (``:8001``), tunnelled with a config that proxied ``/api`` and ``/media`` to the backend. Every request to the ``*.trycloudflare.com`` URL returned ****403 Forbidden**** with ``Server: cloudflare`` / ``CF-Ray``. Local origins returned 200, so the problem was ****Cloudflare's WAF on account-less quick tunnels**** blocking the proxied ``/api`` path — not our code.

****Root cause / fix:**** Cloudflare quick tunnels aggressively challenge requests that hit a separate API proxy path. Solution — ****combine into a single origin****:

- ``main.py``: the FastAPI backend now also serves the built ``dist/`` (SPA + API on the same ``:8001``). A catch-all route serves ``index.html`` for non-API paths, without shadowing ``/auth``, ``/quiz``, ``/admin``, ``/health``, ``/docs``.
- Frontend built with ``VITE_API_URL=/`` so it calls the API at the same origin (no ``/api`` prefix to trip the WAF).
- One ``cloudflared`` tunnel `--url http://localhost:8001`` → single origin → no proxy path → Cloudflare stops blocking.

****Roadblock 2 — catch-all route shadowed ``/health``****

Mounting ``StaticFiles`` at ``/`` intercepted all paths including ``/health`` and ``/admin/token`` (404s). Fix: replaced the ``/`` mount with explicit ``/`` and ``/{full_path:path}`` routes registered ****after**** the API routers, so API routes win.

****Result:**** public link works end-to-end through Cloudflare:

- Landing (SPA): HTTP 200
- ``/health``: ok
- Student registration: 200
- Admin login (``/admin/token``): 200
- Full quiz submit with per-question timing: scored correctly

> ****Link churn:**** quick-tunnel URLs are random and change every restart. The
> current live link is recorded in the chat; when it dies, say "relaunch trivia."

7. Verification checklist (as of 2026-08-19)

Check	Result
<code>`vite build`</code> (frontend)	exit 0 ■
Backend <code>`/health`</code>	ok ■
SPA served at <code>`/`</code> (same origin)	200 ■
Student register	200 ■
Quiz pack (no answer leak)	■

Quiz submit w/ per-question speed bonus	correct scoring ■
Forgot-password (reset question → verify → login)	full flow 200 ■
User wipe (admins + questions kept)	4 admins, 669 Qs ■
Public Cloudflare link (SPA + API)	200 through tunnel ■
Walking mascots + letter interaction	renders, no error ■
Bubble auto-hides + hidden while walking	implemented ■

8. Known limitations / open items

- **Hosting is temporary** — Cloudflare quick tunnel, not a permanent deployment.

A permanent option (Cloudflare Pages + Worker for the API, or a VPS) needs a Cloudflare account or a different host.

- **CGNAT/Starlink** — the machine is behind CGNAT, so only outbound tunnels

(cloudflared) work; no port-forwarding.

- **DB path fragility** — `config.py` uses sqlite:///./trivia.db` (relative to CWD). Launch the backend from backend/` so it hits backend/trivia.db`. An absolute path would make this launch-location-independent.`

• Per the project scope, Phase 1 still omits adaptive difficulty, offline packs, badges, and school-code-gated competition.

9. How we overcame each roadblock (one-line index)

#	Roadblock	Resolution
1	Blank page after Landing edit	Missing <code>`api`</code> import → added <code>`import { api } from '../api.js`</code> (named, not default)
2	Stale cached localhost tab	Restarted Vite, cleared <code>`.vite`</code> cache, hard-reload
3	Cloudflare 403 on quick tunnel	Combined frontend+API into one origin (<code>`:8001`</code>); no <code>`.api`</code> proxy path
4	Catch-all route shadowed <code>`.health`</code>	Used explicit <code>`. / + `/{full_path:path}`</code> routes after API routers
5	"Start again" on slow answers	No restart existed; fixed scoring to per-question speed bonus, never penalise
6	Bubble stayed forever / overlapped	Auto-hide after ~4s; moved above head; hidden while walking
7	Exposed admin password on login	Removed the credentials hint from <code>`.Admin.jsx`</code>

*Appendix — Local access: backend `http://127.0.0.1:8001` (serves SPA + API),
frontend dev `http://localhost:5173`. API docs at `http://127.0.0.1:8001/docs`.

Public (temporary): Cloudflare quick tunnel, URL in chat.*